UNITÀ 2 · PROGETTARE E UTILIZZARE WEB SERVICE · LEZIONE 1

I sistemi distribuiti

Tanti computer indipendenti che devono sembrare uno solo, parlando attraverso una rete che a volte perde, ritarda e si spezza.

5G TPSI Modelli, middleware, fallacie, teorema CAP

PER COMINCIARE

L'ultima volta che un'app non funzionava, di chi era la colpa?

SONDAGGIO

- **A** Del mio telefono
- **B** Della rete: Wi-Fi o 4G/5G
- **C** Dei server dell'app
- **D** Di un servizio di un'altra azienda, usato dall'app

MENTI CHE CI HANNO PRECEDUTO

La definizione più famosa è una battuta

> **Un sistema distribuito è un sistema in cui il guasto di un computer di cui non conoscevi nemmeno l'esistenza può rendere inutilizzabile il tuo computer.**

Leslie Lamport, informatico statunitense, in un'email ai colleghi del 28 maggio 1987

Il tempo

Nel 1978 spiega come ordinare gli eventi senza un orologio comune: gli **orologi logici**.

Il consenso

Inventa **Paxos**: come far decidere la stessa cosa a nodi che possono guastarsi.

Il premio

Turing Award 2013. E sì: ha creato anche **LaTeX**.

DEFINIZIONE

Molti computer, un solo sistema

Un sistema distribuito è un insieme di computer indipendenti che appare ai suoi utenti come un unico sistema coerente.

Andrew Tanenbaum e Maarten van Steen, *Distributed Systems*

- Ogni nodo ha la sua CPU, la sua memoria, il suo orologio.
- Non c'è memoria condivisa: i nodi comunicano solo **scambiandosi messaggi** in rete.
- Un nodo può guastarsi mentre gli altri continuano: **guasto parziale**.

Esempi che usi ogni giorno

- Il Web e i motori di ricerca
- WhatsApp, Instagram, Spotify
- Il circuito dei bancomat
- I giochi online multiplayer
- Il DNS di Internet

PERCHÉ DISTRIBUIRE

Quattro buoni motivi, quattro nuovi problemi

Vantaggio	Esempio	Il prezzo da pagare
Scalabilità	Aggiungo server quando crescono gli utenti	Coordinare tanti nodi
Affidabilità	Dati replicati in tre data center	Tenere le copie coerenti
Condivisione	Una stampante, un database, una GPU per molti	Accessi concorrenti, sicurezza
Vicinanza	Una CDN serve i video da Milano, non dalla California	Latenza variabile, rete inaffidabile

> Scalare **in verticale** = un server più potente. Scalare **in orizzontale** = più server uguali dietro un bilanciatore di carico.

METTITI ALLA PROVA

Il sito delle iscrizioni crolla il primo giorno. Quale scelta lo rende anche più affidabile?

QUIZ

- **A** Un server nuovo con il quadruplo di RAM e CPU
- **B** Una connessione in fibra più veloce per il server
- **C** Quattro server uguali dietro un bilanciatore di carico
- **D** Spostare il database sullo stesso server dell'applicazione

LE TRASPARENZE

Che cosa il sistema nasconde all'utente

Accesso	Mille utenti insieme, sembri l'unico
Posizione	Leggi un file locale o remoto con le stesse operazioni
Migrazione	Ci sono tre copie, ma ne vedi una
Replicazione	Il servizio cambia server mentre lo usi
Guasto	Un nodo cade e tu non te ne accorgi
Concorrenza	Non sai in quale data center è la tua foto

AL COMPUTER · 15 minuti

Quanti server servono per una pagina?

1. Apri il sito di un quotidiano nazionale in Chrome, Edge o Firefox. Premi **F12**, scheda **Network** (Rete).
2. Spunta **Disable cache** e ricarica con **Ctrl+F5**.
3. Leggi in basso il numero di richieste e il tempo totale. Aggiungi la colonna **Domain** (tasto destro sulle intestazioni).
4. Ordina per dominio e conta i domini diversi: quali sono del giornale, quali di CDN, pubblicità, statistiche?
5. Annota tutto in **pagina-distribuita-cognome.txt**.

Controllo: di solito trovi decine o centinaia di richieste verso 10-50 domini diversi. Una sola pagina è già un sistema distribuito: se uno di quei server è lento, la pagina resta "a metà".

MODELLI ARCHITETTURALI

Chi chiede e chi risponde

Client-server

Il client chiede, il server risponde. Il server è centrale: facile da gestire e proteggere, ma va dimensionato bene.

Web, posta, giochi online, home banking

Peer-to-peer

Ogni nodo è insieme client e server. Più nodi ci sono, più risorse ci sono. Difficile da controllare.

BitTorrent, blockchain, aggiornamenti di Windows tra PC vicini

A livelli (multi-tier)

Il server si divide in livelli: **presentazione**, **logica**, **dati**. Ogni livello può scalare da solo.

Browser, server applicativo, database

> Molti sistemi reali sono **ibridi**: le videochiamate usano un server per trovarsi, poi spesso un flusso diretto.

MIDDLEWARE

Lo strato che nasconde la rete al programmatore

Applicazioni distribuite

App del registro elettronico, e-commerce, gioco online

Middleware

RPC e Java RMI · broker di messaggi (RabbitMQ, Kafka) · web service SOAP e REST · gRPC

Sistemi operativi e rete

Windows, Linux, Android · TCP/IP · socket

Il **middleware** è il software che sta tra il sistema operativo e l'applicazione e offre servizi comuni:

- comunicazione tra nodi
- conversione dei dati
- nomi e ricerca dei servizi
- sicurezza e transazioni

Tutta l'Unità 2 è un viaggio tra diversi tipi di middleware.

LE OTTO FALLACIE

Otto cose che i programmatori credono, sbagliando

1. La rete è affidabile.
2. La latenza è zero.
3. La banda è infinita.
4. La rete è sicura.
5. La topologia non cambia.
6. C'è un solo amministratore.
7. Il costo del trasporto è zero.
8. La rete è omogenea.

Le "fallacie del calcolo distribuito": elenco nato in Sun Microsystems negli anni '90, reso famoso da **L. Peter Deutsch**; l'ottava è di **James Gosling**, il padre di Java.

AL COMPUTER · 20 minuti

La rete non è affidabile: riprova con giudizio

```
var client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(2)).build();
var req = HttpRequest.newBuilder(URI.create(a[0])).build();
long attesa = 500;
for (int t = 1; t <= 4; t++) {
  try {
    var r = client.send(req, HttpResponse.BodyHandlers.ofString());
    System.out.println("Tentativo " + t + ": HTTP " + r.statusCode());
    return;
  } catch (java.io.IOException e) {
    System.out.println("Tentativo " + t + " fallito: "
        + e.getClass().getSimpleName() + ", attendo " + attesa);
    Thread.sleep(attesa);
    attesa *= 2;   // backoff esponenziale
  }
}
System.out.println("Irraggiungibile: uso i dati in cache");
```

1. Crea **Riprova.java**: il codice va nel `main`, con le import di `java.net`, `java.net.http` e `java.time`.
2. Esegui `java Riprova.java URL` con questi tre URL:
   `https://www.wikipedia.org`
   `http://localhost:9`
   `http://10.255.255.1`

Controllo: Wikipedia risponde subito. Con la porta 9 ricevi **ConnectException** immediata, con 10.255.255.1 un **timeout** dopo 2 secondi.

IL TEOREMA CAP

Quando la rete si spezza, devi scegliere

C

Consistency

Tutti i nodi vedono lo stesso dato nello stesso momento.

A

Availability

Ogni richiesta riceve una risposta, anche se un nodo è irraggiungibile.

P

Partition tolerance

Il sistema continua a funzionare anche se la rete si divide in due parti.

> In rete le **partizioni capitano**. Durante una partizione un nodo può rispondere con un dato forse vecchio (**A**) oppure rifiutarsi finché non è sicuro (**C**). Tutte e due insieme, no.

Congettura di Eric Brewer (2000), dimostrata da Seth Gilbert e Nancy Lynch (2002).

QUIZ A TRABOCCHETTO

Quale frase sul teorema CAP è corretta?

QUIZ

A Si scelgono liberamente due proprietà su tre, per esempio C e A

B Se la rete si divide, bisogna scegliere tra coerenza e disponibilità

C Un database relazionale garantisce sempre tutte e tre

D Con la fibra ottica le partizioni non avvengono più

AL COMPUTER · 15 minuti

La latenza non è zero: misurala

```
curl -o NUL -s -w "connessione %{time_connect} s  totale %{time_total} s\n" https://www.wikipedia.org
```

1. Nel **Prompt dei comandi** esegui il comando 3 volte. In PowerShell scrivi `curl.exe`; su Linux e macOS usa `-o /dev/null`.
2. Ripeti con il sito del tuo comune e con un sito di un altro continente.
3. Calcola il minimo teorico: nella fibra la luce fa circa **200 000 km/s**. Milano–New York sono circa 6 500 km: quanto vale l'andata e ritorno?
4. Salva valori e calcoli in **latenza-cognome.txt**.

Controllo: il minimo teorico per Milano–New York è circa **65 ms** (andata e ritorno). La connessione a un sito vicino richiede pochi millisecondi; se un sito "lontano" risponde in 10 ms, lo sta servendo una CDN in Europa.

OGGI

I sistemi distribuiti più grandi del pianeta





ALL'ESAME · SFIDA A SQUADRE

Progetta in 6 minuti: prenotare le aule della scuola

Docenti e studenti prenotano laboratori e palestra da app e browser. Disegnate l'architettura e rispondete:

1. Quali sono i tre livelli e su quali macchine stanno?
2. Che cosa succede se cade il server del database?
3. Due docenti prenotano lo stesso laboratorio nello stesso istante: chi vince?
4. Durante una partizione preferite C o A? Perché?

⏱ 6:00

All'esame una traccia di progetto chiede quasi sempre uno **schema dell'architettura** e la **giustificazione** delle scelte. Un disegno senza motivazioni vale la metà.

SFIDA FINALE

Apri il lucchetto distribuito



- 1ª cifra: quante sono le fallacie del calcolo distribuito
- 2ª cifra: quante proprietà nomina il teorema CAP
- 3ª cifra: l'ultima cifra dell'anno dell'email di Lamport
- 4ª cifra: durante una partizione, quante tra C e A puoi garantire

`codice`

BIGLIETTO D'USCITA

Completa prima di uscire

In un sistema distribuito i nodi non condividono la memoria: comunicano solo scambiandosi _____.

Quando un nodo si guasta e gli altri continuano si parla di guasto _____.

Aggiungere server uguali dietro un bilanciatore significa scalare in _____.

Lo strato software tra sistema operativo e applicazione si chiama _____.

Secondo il teorema CAP, durante una partizione si sceglie tra coerenza e _____.

⏱ 3:00

RIEPILOGO

Cosa portiamo a casa

- Computer indipendenti che sembrano un solo sistema
- Solo messaggi in rete, guasti parziali, nessun orologio comune
- Client-server, peer-to-peer, architetture a livelli
- Il middleware nasconde la rete al programmatore
- Otto fallacie: la rete non è affidabile, né istantanea
- CAP: con una partizione, coerenza o disponibilità

Hai una domanda su una slide? Usa il pulsante **? Domande** in basso: la vedrà il docente e tutta la classe.